

```
In [6]: # Adaptive Boost Regression - Insurance Charges prediction

# Hyper tuning parameter - base_estimator=None, learning_rate=1.0, loss='linear', n_estimators=100, random_state=0)

# r_score = 0.8618083410771146

import pandas as pd
```

```
In [7]: #DATASET COLLECTION

dataset=pd.read_csv("Insurance_pre.csv")
```

```
In [8]: dataset
```

Out[8]:

	age	sex	bmi	children	smoker	charges
0	19	female	27.900	0	yes	16884.92400
1	18	male	33.770	1	no	1725.55230
2	28	male	33.000	3	no	4449.46200
3	33	male	22.705	0	no	21984.47061
4	32	male	28.880	0	no	3866.85520
...	...	...	...	...	...	...
1333	50	male	30.970	3	no	10600.54830
1334	18	female	31.920	0	no	2205.98080
1335	18	female	36.850	0	no	1629.83350
1336	21	female	25.800	0	no	2007.94500
1337	61	female	29.070	0	yes	29141.36030

1338 rows × 6 columns

In [9]: *#PRE PROCESSING*

```
dataset=pd.get_dummies(dataset,drop_first=True)  
dataset
```

Out[9]:

	age	bmi	children	charges	sex_male	smoker_yes
0	19	27.900	0	16884.92400	0	1
1	18	33.770	1	1725.55230	1	0
2	28	33.000	3	4449.46200	1	0
3	33	22.705	0	21984.47061	1	0
4	32	28.880	0	3866.85520	1	0
...	...	...	...	...	...	...
1333	50	30.970	3	10600.54830	1	0
1334	18	31.920	0	2205.98080	0	0
1335	18	36.850	0	1629.83350	0	0
1336	21	25.800	0	2007.94500	0	0
1337	61	29.070	0	29141.36030	0	1

1338 rows × 6 columns

In [10]: dataset. columns

Out[10]: Index(['age', 'bmi', 'children', 'charges', 'sex_male', 'smoker_yes'], dtype='object')

In [11]: *# INPUT AND OUTPUT COLUMNS SPLIT UP*
input= independent

```
independent=dataset[['age', 'bmi', 'children', 'sex_male', 'smoker_yes']]  
independent
```

Out[11]:

	age	bmi	children	sex_male	smoker_yes
0	19	27.900	0	0	1
1	18	33.770	1	1	0
2	28	33.000	3	1	0
3	33	22.705	0	1	0
4	32	28.880	0	1	0
...	...	...	...	...	...
1333	50	30.970	3	1	0
1334	18	31.920	0	0	0
1335	18	36.850	0	0	0
1336	21	25.800	0	0	0
1337	61	29.070	0	0	1

1338 rows × 5 columns

```
In [12]: # output=dependent

dependent=dataset[['charges']]
dependent
```

```
Out[12]:
```

	charges
0	16884.92400
1	1725.55230
2	4449.46200
3	21984.47061
4	3866.85520
...	...
1333	10600.54830
1334	2205.98080
1335	1629.83350
1336	2007.94500
1337	29141.36030

1338 rows × 1 columns

```
In [13]: # TRAINING AND TEST SET SPLIT UP

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test= train_test_split(independent,dependent,te
st_size=0.30,random_state=0)
```

```
In [14]: # STANDARDIZATION

from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)
```

```
In [15]: #USING HYPER TUNING PARAMETERS LIKE KERNEL,C VALUES
from sklearn.ensemble import AdaBoostRegressor
regressor = AdaBoostRegressor(random_state=0, n_estimators=50,learning_rate
=1.0, loss='linear')

regressor.fit(X_train, Y_train)
```

C:\Users\user\Anaconda3\lib\site-packages\sklearn\utils\validation.py:724:
DataConversionWarning: A column-vector y was passed when a 1d array was ex
pected. Please change the shape of y to (n_samples,), for example using r
avel().
y = column_or_1d(y, warn=True)

```
Out[15]: AdaBoostRegressor(base_estimator=None, learning_rate=1.0, loss='linear',
n_estimators=50, random_state=0)
```

```
In [16]: regressor.estimator_weights_
```

```
Out[16]: array([[1.99821604, 0.96588611, 0.692996 , 0.19985193, 0.3683144 ,
                0.15490461, 0.32226472, 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ,
                0.          , 0.          , 0.          , 0.          , 0.          ]])
```

```
In [17]: #regressor.n_features_
regressor.estimator_errors_
```

```
Out[17]: array([[0.11939035, 0.27570125, 0.33336693, 0.45020265, 0.40894838,
                0.4613511 , 0.42012392, 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ,
                1.          , 1.          , 1.          , 1.          , 1.          ]])
```

```
In [18]: #regressor.feature_names_in_
```

```
In [19]: Y_pred=regressor.predict(X_test)
```

```
In [20]: #Evaluation
```

```
from sklearn.metrics import r2_score
r_score=r2_score(Y_test,Y_pred)
```

```
In [21]: r_score
```

```
Out[21]: 0.8618083410771146
```

```
In [22]: # base_estimator=None, learning_rate=1.0, loss='linear',n_estimators=50, ra
         ndom_state=0)      r_score = 0.8618083410771146
         # base_estimator=None, learning_rate=1.0, loss='linear',n_estimators=100, r
         andom_state=0)      r_score = 0.8618083410771146

         # base_estimator=None, learning_rate=2.0, loss='square',n_estimators=50, ra
         ndom_state=0)      r_score = 0.5101725090162179
         # base_estimator=None, learning_rate=1.0, loss='square',n_estimators=100, r
         andom_state=0)      r_score =0.48423069369909855
         # base_estimator=None, learning_rate=1.0, loss='exponential',n_estimators=5
         0, random_state=0)  r_score =0.6238747714263077
         # base_estimator=None, learning_rate=1.0, loss='exponential',n_estimators=1
         00, random_state=0) r_score = 0.5494148599707891
```

```
In [23]: import pickle
         filename="finalized_model_AdaBoost.sav"
```

```
In [24]: pickle.dump(regressor,open(filename,'wb'))
```

```
In [25]: # to load the model  
loaded_model=pickle.load(open("finalized_model_AdaBoost.sav",'rb'))
```

```
In [28]: result=loaded_model.predict([[18,31.92,0,0,0]])
```

```
In [29]: result
```

```
Out[29]: array([16708.77857483])
```

```
In [ ]:
```